Record of Training

MicrosoftLearn – Build Web Apps with Blazor

Course Material

Blazor is a complete frontend web framework based on the foundation of .Net. It's based on the languages,

- C#
- CSS
- HTML

You build web apps using reusable components that can be run from both the client and server side.

Blazor components

Components in Blazor are reusable pieces of web UI. It encapsulates both its rendering and UI event handling logic. It houses many built-in components such as form handling, user input validation, displaying large data sets, authentication and authorization. You can also build and share custom built components.

You author Blazor components using razor syntax which is a mixture of HTML, CSS and C#. A razor file contains plain HTML and then C# to define any rendering logic. Razor files are then compiled into C# classes that encapsulate a components rendering logic.

UI Event Handling and Data Binding

Interactive Blazor components can handle standard web UI interactions using C# event handlers.

So, in the following example of code, most of the text is in HTML with the logic set using C# at the @code section

```
<h1>Counter</h1>


<p role="status">Current count: @currentCount</p>


<button class="btn btn-primary"
@onclick="IncrementCount">Click me</button>


@code {
   private int currentCount = 0;


   private void IncrementCount()
   {
      currentCount++;
   }
}
```

The @ symbol is used to insert C# code and to tell Blazor that you are using C# logic.

You can separate your C# logic from your razor pages in fact it is best practice to do so. You just must register the service in your main file and insert the using namespace at the top of your code.

🧩 Project Structure

```
/Pages
    Counter.razor
    Counter.razor.cs
/Services
    CounterService.cs
Program.cs
```

1️⃣ Service (separate C# logic)

Services/CounterService.cs

```C#
namespace MyBlazorApp.Services
{
    public class CounterService
    {
        public int Increment(int value)
        {
            return value + 1;
        }
    }
}
```

2️⃣ Register the Service

Program.cs

```C#
using MyBlazorApp.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorComponents();
builder.Services.AddScoped<CounterService>();

var app = builder.Build();

app.MapRazorComponents<App>();

app.Run();
```

Key points from module assessment

- Blazor is a modern frontend web framework based on HTML, CSS and C#.
- Blazor reusable components typically use razor language which is mix of C# and HTML.
- You can use existing Javascript libraries with Blazor
- Benefits of using Blazor include: Build web UI fast with reusable components, Support for server and client-side rendering and building the frontend and backend with one development stack.

Module: Build your first web app with Blazor

Configuring the environment, you can use command line tools to create a new Blazor project with,

dotnet new blazor

You can then use the dotnet build and run commands to build and run the project. You can use the command,

dotnet watch

command to build and run a project and then automatically apply code changes as they are made.

Building a new Blazor app imports folder structure and necessary dependencies automatically and sets up you page.

Folders and files created include:

- *Program.cs* is the entry point for the app that starts the server and where you configure the app services and middleware.
- *App.razor* is the root component for the app.
- *Routes.razor* configures the Blazor router.
- The *Components/Pages* directory contains some example web pages for the app.
- *BlazorApp.csproj* defines the app project and its dependencies and can be viewed by double-clicking the project node in the Solution Explorer.
- The *launchSettings.json* file inside the Properties directory defines different profile settings for the local development environment. A port number is automatically assigned at project creation and saved on this file.

Razor Pages

When building a Blazor project, you will get a few example razor pages and a home.razor page, this is your websites home page (landing page). In Blazor, a Razor file defines a reusable component that makes up a portion of the app UI. Components define what HTML to render and how to handle user events.

At compile time, each Razor component is built into a C# class.

Using Components

To use a component from another component, you can add an HTML-style tag with a name that matches the component name.

For example, if you have a component named *MyButton.razor*, you can add a MyButton component to another component by adding a <MyButton /> tag.

To reference another component from another page. You can place the name of the Razor page in the code

<PageName />

For example, in the basic project files you have a counter page named Counter.razor. This page represents one component (the counter logic and display). To display this on the home page you simply insert into the home page,

<Counter />

Then the counter component will display on your home page.

Parameters

You can add parameters to your C# code by adding the code

[Parameter]

Above the parameter you want to add, no brackets are required to encapsulate the code,

[Parameter]

public int IncrementAmount { get; set; };